

เอกสารวางแผนพัฒนา (Development Planning Documentation)

ระบบฟอร์มรับข้อมูลผู้ป่วย และหน้าจอมอนิเตอร์เจ้าหน้าที่แบบเรียลไทม์

งานทดสอบตำแหน่ง Front-end Developer · Next.js · TailwindCSS · Socket.IO

1. โครงสร้างโปรเจกต์ (Project Structure)

โปรเจกต์นี้แบ่งแยกส่วน page routes, UI component ที่นำกลับมาใช้ซ้ำได้ และ logic ที่ใช้ร่วมกัน ออกจากกันอย่างชัดเจน เพื่อให้ตรรกะการทำงานจริงอยู่ในไฟล์ที่ทดสอบและนำกลับมาใช้ใหม่ได้ง่าย แทนที่จะฝังอยู่ในไฟล์หน้า (page) โดยตรง

```
agnos-patient-system/
├── server.js                # Custom Node server: Next.js + Socket.IO
├── pages/
│   ├── _app.js             # โหลด global style ให้ทุกหน้า
│   ├── index.js            # เส้นทาง "/" - หน้าฟอร์มผู้ป่วย
│   └── staff.js            # เส้นทาง "/staff" - หน้าจอเจ้าหน้าที่
├── components/
│   ├── PatientForm.jsx     # ฟอร์มผู้ป่วยและขั้นตอนต่างๆ
│   ├── FormField.jsx       # เรนเดอร์ input แต่ละช่อง (text/select/date/...)
│   ├── StaffDashboard.jsx  # กริด, ตัวกรอง, ช่องค้นหา ผังเจ้าหน้าที่
│   ├── PatientCard.jsx     # การ์ดแสดงข้อมูลผู้ป่วยหนึ่งคน
│   └── StatusPulse.jsx     # จุดไฟกะพริบแสดงสถานะ
├── lib/
│   ├── formSchema.js       # แหล่งข้อมูลกลาง: รายชื่อฟิลด์ + กฎ validation
│   ├── patientSession.js   # เก็บ patient id แบบถาวรใน localStorage
│   ├── useSocket.js        # การเชื่อมต่อ Socket.IO ที่ใช้ร่วมกัน (React hook)
│   └── time.js             # ฟังก์ชันแปลงเวลาเป็น "x วินาทีที่แล้ว"
├── styles/globals.css      # Tailwind layer, ฟอนต์, focus/motion rules
├── tailwind.config.js      # ชุดโทเคนดีไซน์ (สี, ตัวอักษร, แอนิเมชัน)
└── README.md
```

เหตุผลที่แบ่งโครงสร้างแบบนี้: ไฟล์ในโฟลเดอร์ pages/ จะมีแค่ <Head> กับ component เดียวเท่านั้น ส่วน logic จริงทั้งหมดอยู่ใน components/ และ lib/ ไฟล์ formSchema.js เป็นแหล่งข้อมูลกลางเพียงจุดเดียวสำหรับชื่อฟิลด์ บ้ายกำกับ และกฎ validation ทั้งฝั่งฟอร์มผู้ป่วยและฝั่งหน้าจอเจ้าหน้าที่อ่านข้อมูลจากจุดเดียวกันนี้ ทำให้ทั้งสองฝั่งไม่มีทางไม่ตรงกันเมื่อมีการเพิ่มหรือแก้ไขฟิลด์

2. การออกแบบ (Design)

ฝั่งฟอร์มผู้ป่วยต้องการความรู้สึกสงบและอุ่นใจ เพราะผู้ป่วยบางคนอาจรอกฟอร์มขณะไม่สบายหรือกังวลใจ ส่วนฝั่งหน้าจอเจ้าหน้าที่ต้องการความรู้สึกเหมือนศูนย์มอনিเตอร์ คือแน่น อ่านง่าย และมองเห็นภาพรวมได้ในพริบตาจากระยะไกล ความรู้สึกสองแบบนี้เป็นตัวกำหนดแนวทางภาพที่ต่างกัน แต่ยังใช้ระบบดีไซน์เดียวกันเป็นฐาน

2.1 ระบบภาพและสี

- **ฟอร์มผู้ป่วย:** พื้นหลังโทนกระดาสีอ่อนๆ (#FBF9F6) ตัวอักษรสีเขียวอมฟ้าเข้ม และสีเน้นโทนดินเผา/โคลาย (clay) เพียงสีเดียวสำหรับปุ่มหลัก เพื่อให้ประสบการณ์ดูสงบ ไม่รู้สึกเหมือนแบบฟอร์มโรงพยาบาลแข็งทื่อ
- **หน้าจอบริการ:** พื้นหลังโทนดำนุ่มเทาเข้ม ทำให้รู้สึกเหมือนแดชบอร์ดมอนิเตอร์ทันที ไม่ใช่ฟอร์มกรอกข้อมูล โดยยังใช้สีเน้นชุดเดียวกัน (ink/clay/sage) เพื่อความต่อเนื่องระหว่างสองหน้าจอ
- **องค์ประกอบเอกลักษณ์:** จุดไฟกะพริบเล็กๆ บนการ์ดผู้ป่วยแต่ละคน — เป็นวงกระพือเมื่อ "กำลังกรอก", ที่บสีเขียว sage เมื่อ "ส่งแล้ว", และสีจางลงเมื่อ "ไม่ได้ใช้งาน" — ทำให้เจ้าหน้าที่ที่กวาดสายตาดูทั้งหน้าจอแล้วรู้ทันทีว่าใครต้องให้ความสนใจ โดยไม่ต้องอ่านตัวหนังสือเลย
- **ตัวอักษร:** **Fraunces** (เซอริฟ) สำหรับหัวข้อ ให้ความรู้สึกอบอุ่นแบบมนุษย์แต่ยังคงเป็นคลินิกนิตย, **Inter** สำหรับเนื้อหาและข้อมูล เพราะอ่านง่ายแม้ตัวเล็กในกริดที่แน่น, **IBM Plex Mono** สำหรับรหัสอ้างอิงผู้ป่วย เพื่อให้แยกออกจากชื่อคนได้ชัดเจนด้วยรูปแบบตัวอักษร

2.2 การตัดสินใจ UI/UX ตามขนาดหน้าจอ

หน้าจอ / Breakpoint	การตัดสินใจ
ฟอร์มผู้ป่วย — มือถือ (< 640px)	จัดเป็นคอลัมน์เดียวตลอดทั้งฟอร์ม ไม่มีข้อยกเว้น แบ่งเป็น 3 ขั้นตอนสั้นๆ (ข้อมูลตัวตน → ข้อมูลติดต่อ → ข้อมูลเพิ่มเติม) พร้อม progress bar แทนที่จะให้เลื่อนดูฟอร์มยาวทีเดียว เพื่อไม่ให้รู้สึกหนักใจเมื่อใช้บนจอเล็ก ช่องกรอกข้อมูลใช้แป้นพิมพ์ที่เหมาะสมกับมือถือ (tel, email, date) ผ่าน input type ของ HTML โดยตรง
ฟอร์มผู้ป่วย — เดสก์ท็อป (≥ 640px)	ยังคงเป็นคอลัมน์เดียวเช่นเดิม แต่จำกัดความกว้างให้อ่านสบายตา (max-w-lg) และจัดกึ่งกลางหน้าจอ เพราะฟอร์มไม่ได้ดีขึ้นเมื่อยืดช่องกรอกข้อมูลให้กว้างเต็มจอ จึงไม่ใช่เลยเอาต์หลายคอลัมน์ในส่วนนี้
หน้าจอบริการ — มือถือ (< 640px)	การเรียงผู้ป่วยเรียงเป็นคอลัมน์เดียว (grid-cols-1) เพื่อให้ทุกฟิลด์อ่านได้ง่ายโดยไม่ต้องเลื่อนจอในแนวนอน
หน้าจอบริการ — แท็บเล็ต (sm: ≥ 640px)	กริดปรับเป็น 2 คอลัมน์ (sm:grid-cols-2) ให้เห็นภาพรวมได้มากขึ้นโดยที่การคลิกยังไม่แคบจนอ่านยาก
หน้าจอบริการ — เดสก์ท็อป (xl: ≥ 1280px)	กริดขยายเป็น 3 คอลัมน์ (xl:grid-cols-3) เพื่อให้มองเห็นผู้ป่วยได้พร้อมกันมากขึ้นบนจอมอนิเตอร์ขนาดใหญ่ โดยแต่ละการดยังคงความกว้างที่อ่านสบาย ไม่ถูกยัดจนบางเกินไป

2.3 มาตรฐานขั้นต่ำด้านการเข้าถึง (Accessibility)

- มีกรอบ focus ที่มองเห็นได้ชัดเจนบนทุกอิลิเมนต์ที่กดหรือเลือกได้ (:focus-visible)
- รองรับการตั้งค่า prefers-reduced-motion — แอนิเมชันจุดไฟกะพริบและการเปลี่ยนสถานะจะถูกปิดสำหรับผู้ใช้ที่ตั้งค่าลดการเคลื่อนไหว
- รักษาความคมชัดของสีตัวอักษรให้เพียงพอ ทั้งในธีมสว่าง (ฝั่งผู้ป่วย) และธีมมืด (ฝั่งเจ้าหน้าที่)

3. สถาปัตยกรรม Component (Component Architecture)

การไหลของ state ในแต่ละหน้าเป็นทิศทางเดียว: มีเพียง PatientForm และ StaffDashboard เท่านั้นที่คุยกับ socket โดยตรง ส่วน component ที่อยู่ภายใต้ทั้งสองตัวนี้เป็น component เชิงแสดงผลล้วนๆ (รับ props เข้ามา แสดง UI ออกไป) ไม่รู้จัก socket เลย

Component	หน้าที่
PatientForm	ดูแล state ทั้งหมดของฟอร์ม (ขั้นตอนปัจจุบัน, ค่าที่กรอก, ข้อผิดพลาดจาก validation) หน่วงเวลา (debounce ~350ms) ก่อนส่งข้อมูลที่เปลี่ยนแปลงผ่าน socket, ตรวจสอบความถูกต้องของแต่ละขั้นตอนก่อนอนุญาตให้กด "ถัดไป" และตรวจสอบทุกฟิลด์อีกครั้งตอนกดส่ง เมื่อส่งสำเร็จจะแสดงหน้ายืนยัน
FormField	เป็น component เชิงแสดงผลส่วนๆ รับนิยามฟิลด์ ค่าปัจจุบัน และข้อผิดพลาด (ถ้ามี) มาเรนเดอร์ input ให้ถูกประเภท (text / select / date / textarea / tel / email) พร้อมข้อความแจ้งเตือน ไม่รู้เรื่อง socket หรือขั้นตอนของฟอร์มเลย
StaffDashboard	เข้าร่วมห้อง socket ชื่อ "staff" และเก็บข้อมูลผู้ป่วยทั้งหมดไว้ใน state รูปแบบ { [patientId]: record } คำนวณรายการที่กรองแล้ว/เรียงลำดับแล้ว และจำนวนผู้ป่วยแยกตามสถานะ ดูแล state ของช่องค้นหาและตัวกรองสถานะ
PatientCard	การ์ดแสดงข้อมูลผู้ป่วยหนึ่งคน แสดงชื่อ, ค่าฟิลด์ปัจจุบัน, จุดไฟแสดงสถานะ และเวลาล่าสุดที่มีการเคลื่อนไหวแบบสัมผัส (เช่น "5 วินาทีที่แล้ว") มีการ re-render เป็นระยะเพื่อให้เวลาอัปเดตอยู่เสมอ
StatusPulse	component เชิงแสดงผล แปลงค่าสถานะ (filling / submitted / inactive) เป็นสี บ้ายข้อความ และกำหนดว่าจะมีแอนิเมชันกะพริบหรือไม่

4. ขั้นตอนการซิงค์ข้อมูลแบบเรียลไทม์ (Real-Time Synchronization Flow)

ระบบเรียลไทม์ใช้ Socket.IO ทำงานอยู่ภายใน custom Node server (server.js) ที่ครอบ Next.js ไว้อีกชั้น —

จำเป็นต้องทำแบบนี้เพราะรายการการเชื่อมต่อและข้อมูลผู้ป่วยที่เก็บไว้ในหน่วยความจำ ไม่สามารถอยู่ใน serverless function ที่ไม่มีสถานะ (stateless) ได้

ลำดับขั้นตอน

- 1. ผู้ป่วยเปิดหน้า "/" — เบราว์เซอร์จะอ่าน (หรือสร้างใหม่) patientId แบบถาวรจาก localStorage แล้วส่งอีเวนต์ patient:join ฟังก์ชันเซิร์ฟเวอร์จะสร้าง record ในหน่วยความจำสำหรับ id นั้นถ้ายังไม่มี และกระจายข้อมูลไปยังห้อง staff ทันที ทำให้หน้าจอเจ้าหน้าที่ที่เปิดอยู่แล้วเห็นผู้ป่วยรายใหม่ทันทีที่เข้ามา แม้จะยังไม่ได้กรอกฟิลด์ใดเลย
- 2. ผู้ป่วยพิมพ์ข้อมูล — ทุกครั้งที่พิมพ์ ค่าจะอัปเดตใน React state ทันทีเพื่อให้รู้สึกลื่นไหล และนำฟิลด์ที่เปลี่ยนไปเข้าคิวไว้ก่อน จากนั้นใช้การหน่วงเวลา (debounce) 350 มิลลิวินาที รวมการพิมพ์ที่รวดเร็วให้เป็นการส่งอีเวนต์ patient:update เพียงครั้งเดียว แทนที่จะส่งข้อมูลทุกครั้งที่เกิดเป็นพิมพ์
- 3. เซิร์ฟเวอร์รับอีเวนต์ patient:update — นำฟิลด์ที่เปลี่ยนไปผสานเข้ากับ record ของผู้ป่วยคนนั้น ตั้งค่าสถานะเป็น "filling" อัปเดตเวลา lastActive แล้วกระจาย record ทั้งหมดไปยังทุกไคลเอนต์ในห้อง staff (io.to("staff").emit(...))
- 4. หน้าจอเจ้าหน้าที่รับข้อมูลที่กระจายมา — นำไปผสานเข้ากับ map ข้อมูลในเครื่องโดยอ้างอิงจาก patient id ทำให้ React re-render เฉพาะ PatientCard ของผู้ป่วยที่ข้อมูลเปลี่ยนจริงเท่านั้น
- 5. การตรวจจับสถานะไม่ได้ใช้งาน (Inactive) — หลังจากทุกการอัปเดต เซิร์ฟเวอร์จะตั้งเวลาไว้ตรวจสอบอีกครั้งใน ~12 วินาทีถัดไป หากไม่มีการอัปเดตใหม่เข้ามาในช่วงเวลานั้น (และผู้ป่วยยังไม่ได้กดส่ง) สถานะจะเปลี่ยนเป็น "inactive" และแจ้งเตือนฝั่งเจ้าหน้าที่โดยอัตโนมัติ หากมีการอัปเดตใหม่เข้ามา สถานะจะกลับเป็น "filling" ทันที การตรวจสอบแบบให้ช่วงเวลาผ่อนผันนี้ใช้กับกรณีนี้ socket หลุดการเชื่อมต่อด้วย เพื่อไม่ให้เน็ตสะดุดชั่วคราวทำให้ผู้ป่วยถูกทำเครื่องหมายว่าไม่ได้ใช้งานทันที

- 6. ผู้ป่วยกดส่งฟอร์ม — อีเวนต์ `patient:submit` จะตั้งสถานะเป็น "submitted" และบันทึกเวลา `submittedAt` แล้วกระจายข้อมูลด้วยวิธีเดียวกัน ถือเป็นสถานะสุดท้าย (terminal state) ตัวจับเวลาตรวจสอบความไม่ได้อิงงานจะข้าม record ที่ส่งแล้วไปเลย

มีเพียงเบราว์เซอร์ของผู้ป่วยแต่ละคนเท่านั้นที่เขียนข้อมูลของตนเองได้ (ผู้เขียนเดียวต่อหนึ่ง record)

ในขณะที่การกระจายข้อมูลจะส่งไปถึงทุกไคลเอนต์ฝั่งเจ้าหน้าที่ที่เชื่อมต่อยู่พร้อมกัน — รูปแบบ "ผู้เขียนคนเดียว หลายผู้อ่าน"

นี่เองที่ทำให้เกิดพฤติกรรมเรียลไทม์ตามที่โจทย์ต้องการ โดยไม่ต้องใช้การ *polling* ทั้งสองฝั่งเลย